

6. Distributed Shared Memory

Michael Schöttner

Grundlagen Verteilter Systeme

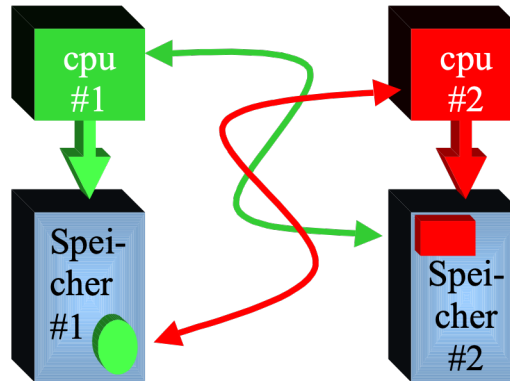
SS 2026

6.1 Allgemeines

- Distributed Shared Memory (DSM)
- Idee: Illusion eines gemeinsamen Speichers für Multicomputer → Kommunikation über den Speicher statt explizitem Message Passing
 - Erste Implementierung IVY von K. Li (1986)
 - Danach zahlreiche Projekte bis ca. 2000
- Ein DSM macht nur Sinn in einem LAN und dieses sollte möglichst schnell sein

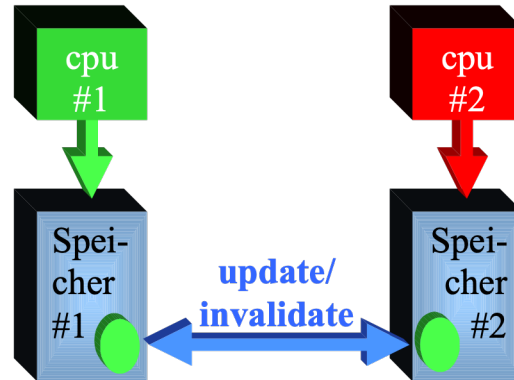
Replikation

- Datenzugriff ohne Replikation:
 - schneller Zugriff im lokalen Speicher
 - langsamer Zugriff im fremden Speicher
 - (auch ohne Replikation ist hier eine Synchronisierung erforderlich)



Replikation

- Datenzugriff mit Replikation:
 - Z.B. auf allen Kopien schreiben & von lokaler Kopie lesen
 - Vermeidet häufige Lesezugriffe über das Netz



- Wann Änderungen für andere Computer sichtbar werden, definiert ein Konsistenzmodelle → siehe später

6.2 Seitenbasierter DSM

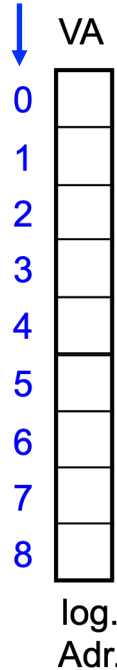
- DSM als lineare Folge von Seiten (i.d.R. 4 KB)
- **Speicherzugriffserkennung erfolgt hier pro Speicherseite**
- Identische logische Adressen auf verschiedenen Knoten verweisen auf die gleichen Daten
 - Typischerweise wird auf jedem Knoten beim Start eines Prozesses ein großer logischer Adressblock reserviert
 - Dieser muss bei allen Knoten die gleiche logische Startadresse haben
 - Nur so sind Zeiger unproblematisch
 - Siehe auch UNIX shared memory mappings

Implementierung des verteilten Zugriffs

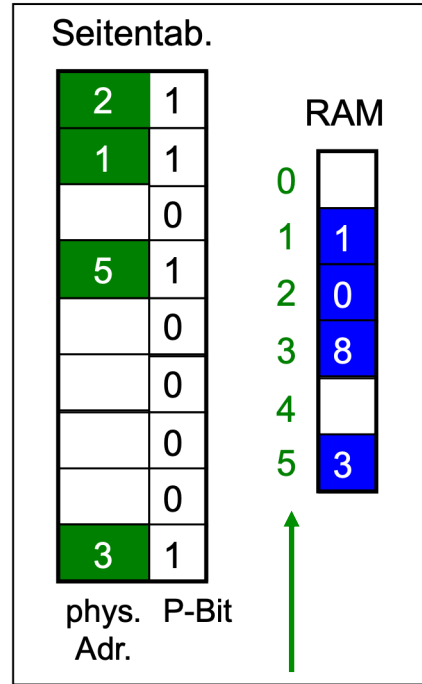
- Erweitern des Paging im Betriebssystem
→ Seiten auch über das Netz ein- und auslagern
- Vorteil: Speicherzugriff transparent und sprachunabhängig erkennbar
- Nachteile:
 - Page-Fault ist vergleichsweise langsam
 - False-Sharing (siehe später)

Ablauf eines entfernten Speicherzugriffs

Seiten-
Nummer

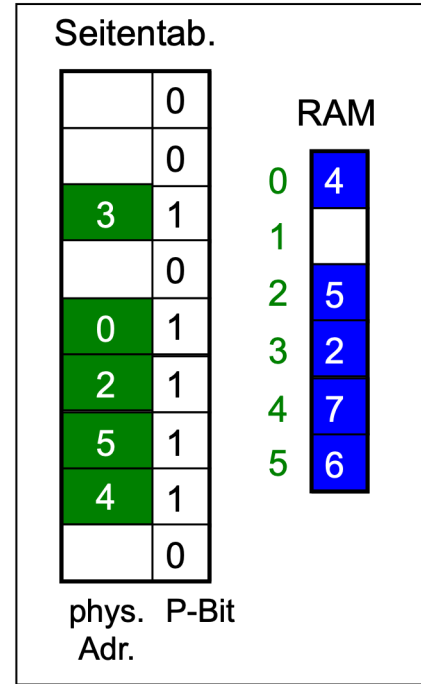


Knoten-1



Kachel-
Nummer

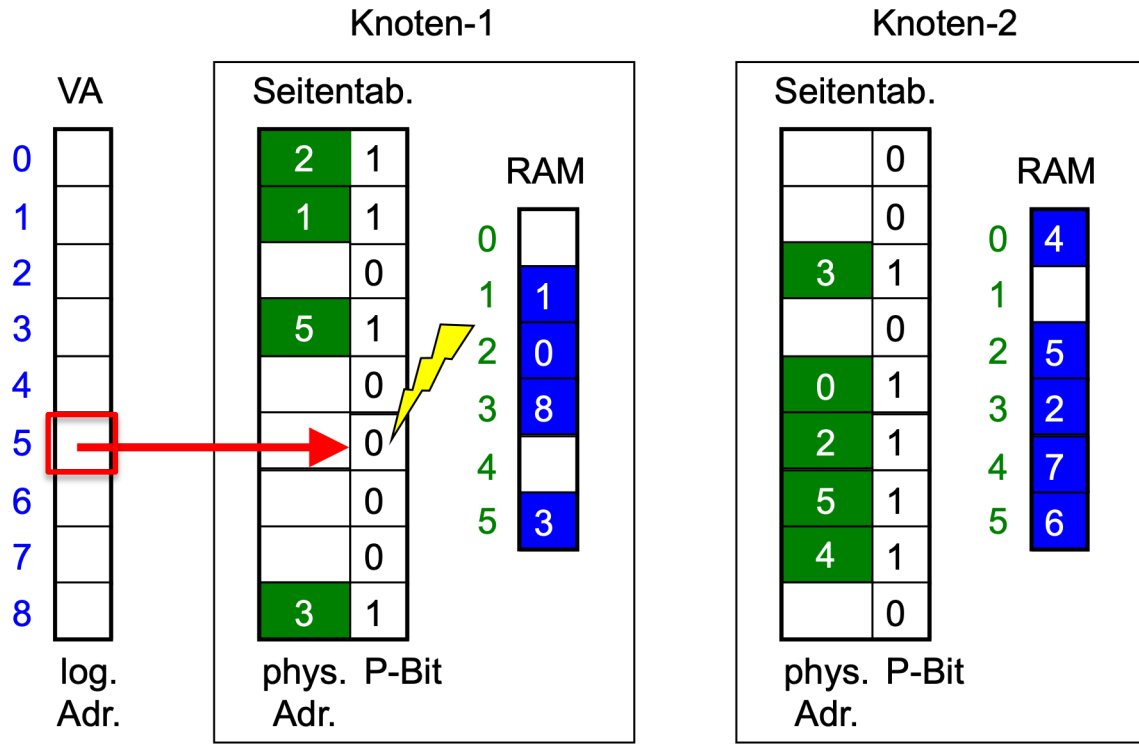
Knoten-2



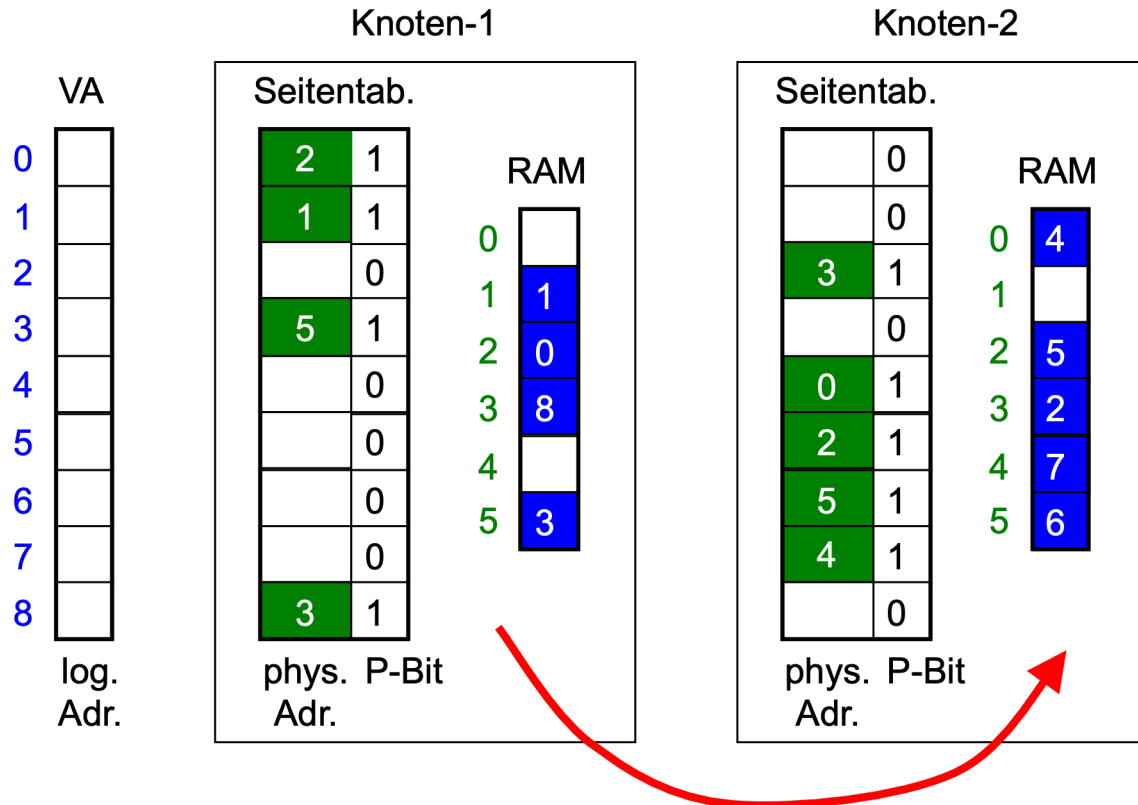
VA = virtueller Adressraum

RAM = physikalischer Speicher

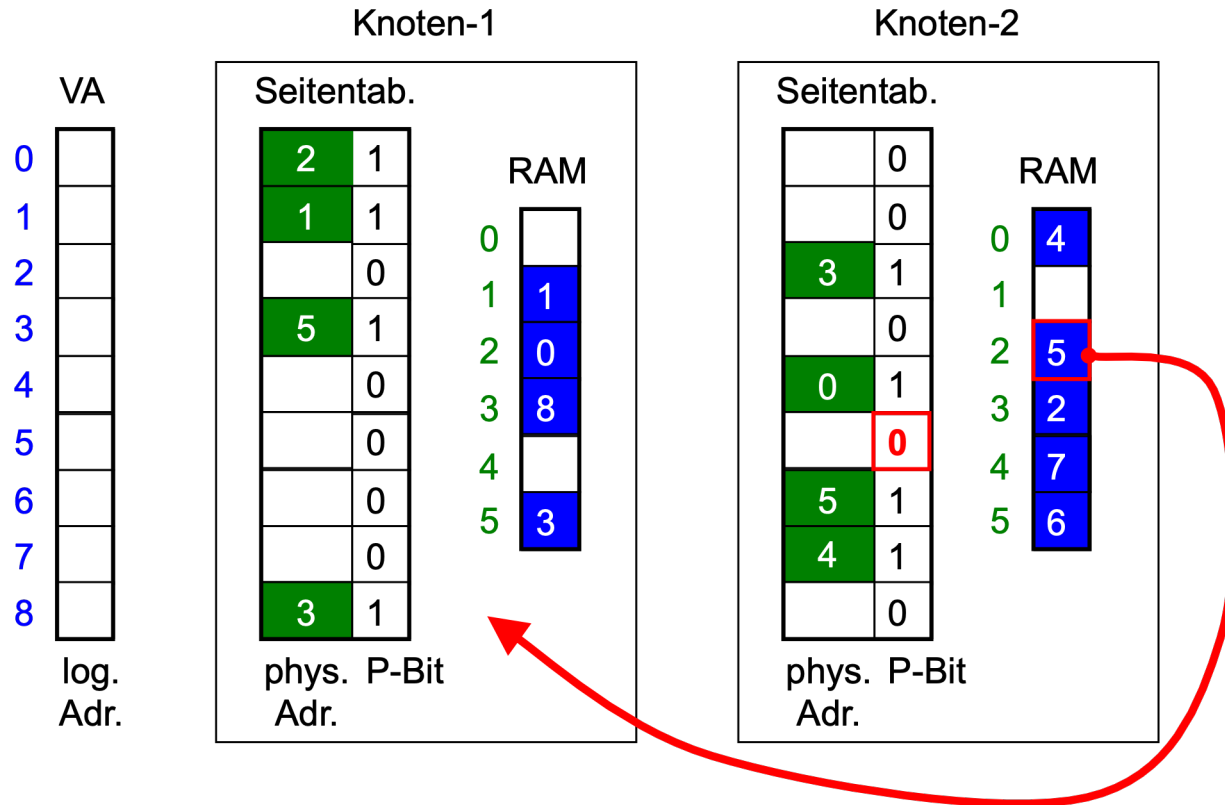
Zugriff auf Seite 5 → nicht präsent → Page Fault



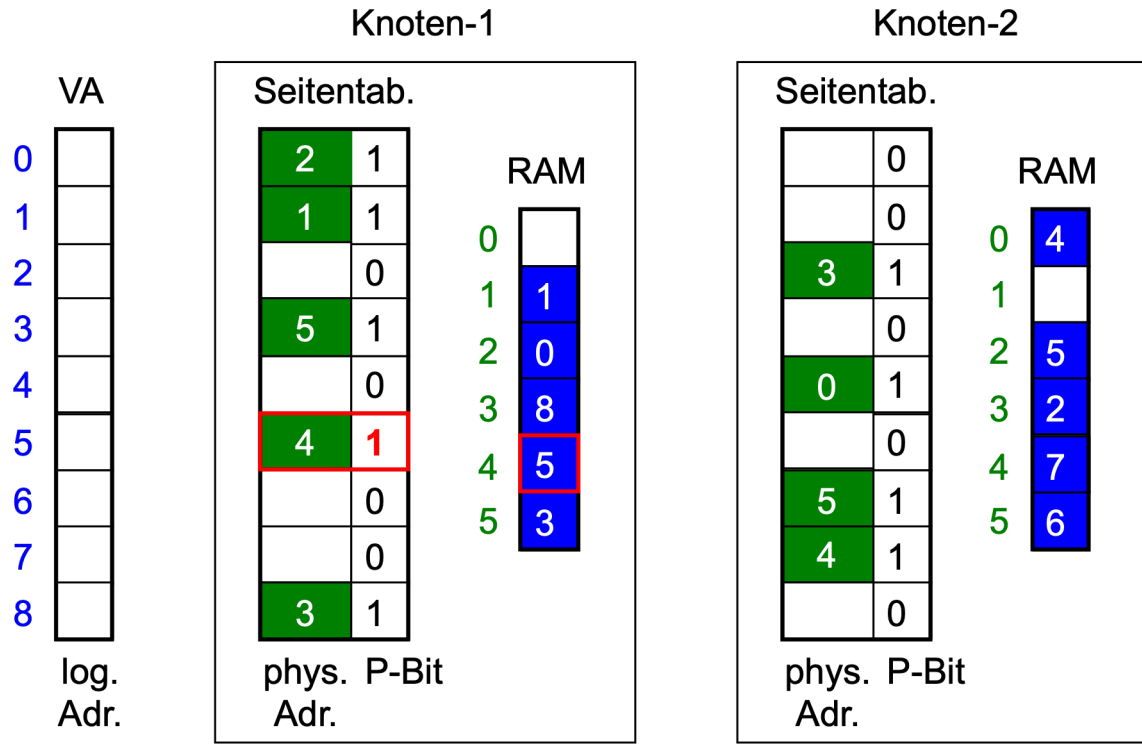
DSM sucht die Seite im Netz und fragt dann an



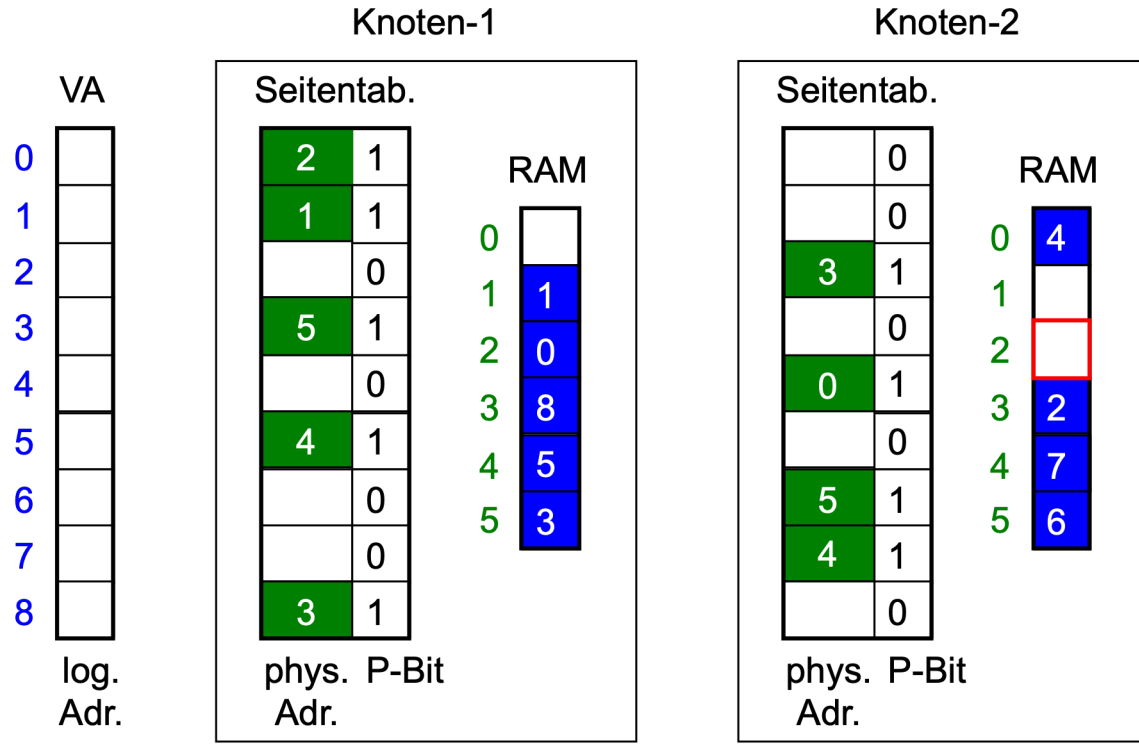
Seite auf „nicht präsent“ setzen & Inhalt versenden



Seite eintragen & Zugriff auf Knoten-1 freigeben



Kachel auf Knoten-2 freigeben



Auffinden von Seiten im Netz

- Bei einem Page-Fault muss die fehlende Seite gefunden werden
- Feste Zuordnung von Adressbereichen zu Knoten
- Anfordern per Multicast
- Verzeichnisdienst: zentral oder verteilt

True-Sharing

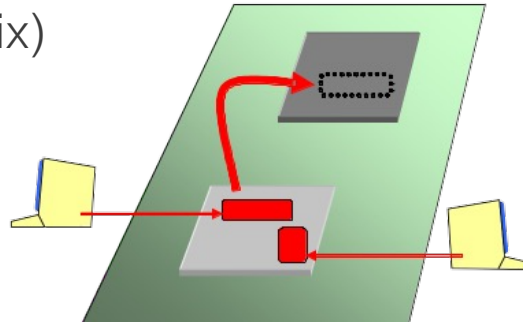
- Zwei oder mehr Objekte liegen auf einer Seite
- Knoten greift ein Objekt zu, wodurch eine Seite aus dem Netz geholt wird.
- Greift nun der Knoten auf weitere Objekte auf dieser Seite zu, so sind diese lokal bereits vorhanden → True-Sharing → Caching-Effekt
- Zugriffe auf benachbarte Speicheradressen sind aufgrund des Lokalisitätsprinzips wahrscheinlich (siehe Betriebssysteme).

False-Sharing

- Grundphänomen in seitenbasierten DSM-Systemen:
 - ≥ 2 Objekte liegen auf derselben Speicherseite
 - Verschiedene Knoten greifen auf unterschiedl. Objekte innerhalb einer Speicherseite zu
 - Hierbei ist ≥ 1 Zugriff schreiben
 - $\rightarrow$ Objekte/Seite wird durch das Netzwerk
 - hin- und her transportiert (u.U. sehr oft $\rightarrow$ Seitenflattern)
- Wichtig: zeitlich abhängiges Phänomen.

False-Sharing

- Lösung 1: Objekte geschickt allozieren → Benutzer muss vorab Zugriffsmuster durchdenken (aufwändig & komplex)
- Lösung 2: Instanzen dynamisch relocieren
Zeigeranpassung notwendig (Plurix)



- Lösung 3: abgeschwächte Konsistenzmodelle
- Lösung 4: kleinere Sharing-Granularität (siehe nächste Folien)
→ kein True-Sharing mehr

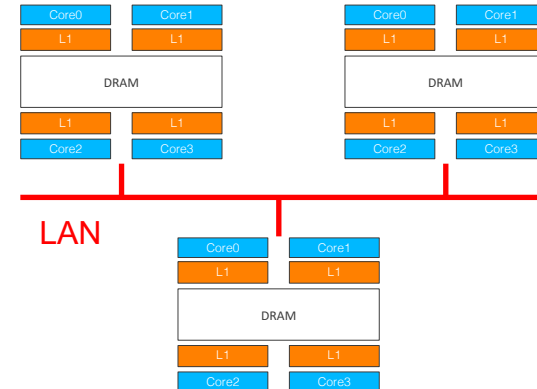
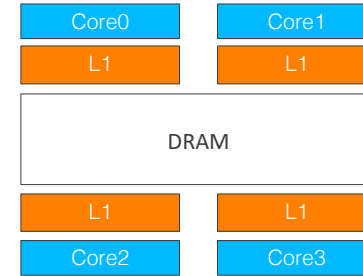
6.3 Objekt-basierter DSM

- Basis ist eine objekt-orientierte Programmiersprache
 - Data-Sharing für Instanzvariablen (auch Referenzen); mit Vererbung, Polymorphie
- **Zugriffserkennung pro Objekt:**
 - Transparent mithilfe spezieller Compiler
 - Generieren bei Zugriff auf Instanzen und Variablen entsprechende Aufrufe an die DSM-Laufzeitbibliothek
- Vermeidet False-Sharing (aber auch kein True-Sharing), aber komplexe Objektstrukturen sind gemeinsam zugänglich

6.4 DSM vs. Multicore

6.4 DSM vs. Multicore

- Multicore
 - Mehrere Cores mit jeweils eigenem L1-Cache
 - Änderungen werden im Prozessor verteilt
 - Realisiert durch die Hardware
- DSM
 - Mehrere CPUs mit jeweils eigenem DRAM
 - Änderungen werden über das LAN verteilt
 - Realisiert durch Software



6.5 Message Passing vs Distributed Shared Memory

Speicherbasierte Kommunikation

- Vereinfachte implizite Kommunikation
- Serialisierung von Objekten entfällt
→ keine Hüllenbildung für Referenzen notwendig
- Konsistenz wird für alle Objekte im DSM garantiert
- Inhärente Replikation ist gute Basis für Fehlertoleranz
- I.d.R. etwas langsamer als Message Passing,
aber einfacher zu programmieren

Nachrichtenbasierte Kommunikation

- Serialisierung und Deserialisierung von Objekten
- Gekapselt in Middleware, z.B. gRPC, Web services, ...
- Mit umfangreichen sprachunabhängigen Laufzeitumgebungen
- Aber Konsistenz & Fehlertoleranz von Daten/Objekt ist die Aufgabe des Programmierers
- Bei korrekter und optimaler Programmierung ist die maximale Nebenläufigkeit erzielbar.

6.6 Zusammenfassung

6.5 Zusammenfassung

- DSM ist eine alte Idee verteilten Speicher transparent zu aggregieren
 - Leider skaliert ein DSM nicht gut, außer man schwächt die Konsistenz ab
 - Dadurch geht aber die Transparenz verloren
 - Konsistenzmodelle werden später besprochen
- Die Idee taucht aber immer wieder in der Literatur auf
 - Im Zusammenhang mit InfiniBand RDMA (Remote Direct Memory Access), ab 2010
 - Deutlich schnellere Netzwerke als damals
 - Sowie bei CXL: Disaggregated Shared Memory, ab 2025
 - Speicherbox mit viel DRAM in einem Server-Rack
 - Anderer Server können über den CXL-Bus den entfernten Speicher im gleichen Rack nutzen → ca. 500ns Zugriffszeit
 - Caches mehrerer Server werden bei Änderungen durch CXL automatisch synchronisiert